



Unidad 06 – Funciones

Cátedra: Programación en Computación – UTN FRRQ **Docentes:** Longhi Pablo, Talijancic Iván

De qué se trata esta unidad

En el TP de la unidad 05 escribiste programas largos, con bloques parecidos repetidos, y el último te dejó con cuatro variables actualizándose en un mismo bucle. Te dijimos que hoy iban a entrar en la mitad de líneas.

Hoy se paga esa promesa.

Al terminar vas a poder:

-  Empaquetar un pedazo de código con un **nombre** y reutilizarlo
-  Pasarle **datos** con parámetros y recibir un **resultado** con `return`
-  Entender por qué las variables de adentro **no se ven** desde afuera
-  Documentar una función con **JSDoc**, como se pide en el curso
-  Generar números aleatorios con `Math.random()` y construir `rndInt(min, max)`
-  Convertir un programa largo en uno corto y legible

 `rndInt()` es la herramienta que más vas a usar de acá en adelante. Con ella podés generar cien mediciones sin tipear ninguna, y en la unidad 07 vas a llenar vectores enteros con una línea.

1. El problema: lo mismo, tres veces

Un programa que verifica tres tableros:

```
const measured1 = 372.5
const deviation1 = (measured1 - 380) / 380 * 100
console.log(`Tablero 1: ${deviation1.toFixed(2)} %`)

const measured2 = 391
const deviation2 = (measured2 - 380) / 380 * 100
console.log(`Tablero 2: ${deviation2.toFixed(2)} %`)

const measured3 = 405
const deviation3 = (measured3 - 380) / 380 * 100
console.log(`Tablero 3: ${deviation3.toFixed(2)} %`)
```

La cuenta es **la misma tres veces**. Y con eso vienen tres problemas:

1. Si la fórmula cambia, hay que corregirla en **tres lugares** — y te vas a olvidar de uno
2. El `380` está escrito tres veces
3. Copiar y pegar es exactamente cómo aparecen los errores que vimos en las revisiones de código: alguien copia la línea y se olvida de cambiar un número

Una **función** le pone nombre a esa cuenta y la escribe **una sola vez**.

2. 🏰 Esto ya lo conocés: la función matemática

Antes de la sintaxis, un puente. En Análisis viste cosas así:

$$f(x) = 2x + 1$$

Eso es una **regla que relaciona dos conjuntos**: a cada valor del **dominio** le asigna exactamente un valor de la **imagen**. Tres partes:

- **Dominio** — de dónde salen los valores que entran
- **Regla de asignación** — qué se hace con ellos
- **Imagen** — los valores que salen

En JavaScript, esa misma función se escribe:

```
const f = (x) => 2 * x + 1
```

Y la correspondencia es casi uno a uno:

EN MATEMÁTICA	EN PROGRAMACIÓN
f , el nombre	el nombre de la función
x , la variable independiente	el parámetro
la regla $2x + 1$	el cuerpo de la función
evaluar $f(3)$	llamar <code>f(3)</code>
el valor $f(3) = 7$	lo que entrega el <code>return</code>
el dominio	los valores que tiene sentido pasarle
la imagen	los valores que puede devolver

2.1 Funciones de varias variables

También las viste. Una función de dos variables:

$$P(V, I) = V \times I$$

```
const power = (voltage, current) => voltage * current
```

Mismo concepto: el dominio ahora son **pares** de valores.

2.2 Definir no es evaluar

Escribir $f(x) = 2x + 1$ en el pizarrón **no calcula nada**. Es una definición. Recién cuando escribís $f(3)$ se hace la cuenta.

En programación es idéntico: **declarar una función no la ejecuta**. Recién al llamarla — `f(3)` — corre el cuerpo. Es la misma distinción, con las mismas palabras.

2.3 ⚠ Dónde se rompe la analogía

La comparación es útil, pero no es exacta. Tres diferencias que conviene tener claras:

1. Una función de programación puede no devolver nada.

```
const printHeader = (panelId) => {  
  console.log(`Tablero ${panelId}`)  
}
```

Eso **no es una función matemática**: no le asigna ningún valor a `panelId`. En programación está perfecto y se usa todo el tiempo.

2. Puede tener efectos sobre el mundo.

`console.log()` cambia algo: lo que se ve en pantalla. $f(x) = 2x + 1$ no cambia nada, sólo devuelve un número.

3. Puede devolver algo distinto cada vez.

Y ésta es la más fuerte. En esta misma unidad vas a escribir:

```
rndInt(1, 6) // 4  
rndInt(1, 6) // 1  
rndInt(1, 6) // 6
```

Mismos argumentos, resultados distintos. **Eso no es una función matemática**, porque la definición exige que a cada elemento del dominio le corresponda **un solo** valor.



Las que **sí** cumplen —mismos argumentos, mismo resultado, sin efectos— se llaman **funciones puras**.

`power(380, 4.2)` da `1596` siempre, hoy y dentro de un año.

Son las más fáciles de probar, de razonar y de reutilizar. **Cuando puedas elegir, escribí funciones puras**: dejá el `console.log` y el azar en el programa principal, y que las funciones sólo calculen.

En la cátedra escribimos **arrow functions**. Ésta la ves para reconocerla, no para usarla.

4. 🧠 Parámetros y argumentos

Dos palabras para dos cosas parecidas, y conviene distinguirlas:

- **Parámetro** — el nombre que aparece en la declaración. Es un hueco a llenar.
- **Argumento** — el valor concreto que pasás al llamarla.

```
const power = (voltage, current) => voltage * current
//           ↑ parámetros

console.log(power(380, 4.2))
//           ↑ argumentos
```

```
1596
```

El orden importa. `power(4.2, 380)` da el mismo número acá porque la multiplicación es conmutativa, pero `deviation(380, 372.5)` **no** es lo mismo que `deviation(372.5, 380)`.

4.1 Si faltan argumentos

JavaScript no se queja: el parámetro que falta queda en `undefined`, y la cuenta da `NaN`.

```
const power = (voltage, current) => voltage * current

console.log(power(380))
```

```
NaN
```

🔑 **Otro origen de NaN**. Ya lo viste en la unidad 03 con `parseFloat('cuatro')`. Si una función te devuelve `NaN`, revisá **cuántos argumentos le pasaste**.

5. ↩️ `return`: devolver un resultado

`return` hace dos cosas a la vez:

1. **Devuelve** un valor a quien llamó
2. **Termina** la función ahí mismo: lo que venga después no se ejecuta

```
const classify = (measured) => {
  if (measured < 361) {
    return 'BAJA'
  }
  if (measured <= 399) {
    return 'NORMAL'
  }
  return 'ALTA'
}

console.log(classify(300))
console.log(classify(380))
```

```
BAJA
NORMAL
```

Fijate que **no hacen falta los `else`**: como el `return` corta, si llegaste a la segunda pregunta es porque la primera dio `false`. Es el mismo razonamiento de la unidad 04, ahora con una herramienta más.

5.1 Lo que devuelve hay que usarlo

Un error muy común: llamar a la función y no hacer nada con el resultado.

```
// ❌ La cuenta se hace y se tira
deviation(372.5, 380)

// ✅ Se guarda
const result = deviation(372.5, 380)

// ✅ 0 se usa directo
console.log(deviation(372.5, 380))
```

5.2 Funciones sin `return`

Algunas funciones no calculan nada: sólo **hacen** algo, como imprimir.

```
const printHeader = (panelId) => {
  console.log('_____')
  console.log(`Tablero ${panelId}`)
  console.log('_____')
}

printHeader('TAB-3')
```

Está perfecto que no devuelvan nada. Pero ojo: si intentás usar su resultado, es `undefined`.

```
const x = printHeader('TAB-3')
console.log(x)
```

```
undefined
```

5.3 ⚠️ Qué se puede devolver

En esta cátedra una función devuelve **un valor primitivo** (número, texto, booleano) o, desde la unidad 07, **un vector o una matriz**.

No se devuelven objetos ni JSON. Es una restricción pedagógica y se corrige como error.

Por ahora, si necesitás devolver tres cosas, escribí tres funciones. Desde la **unidad 07** vas a poder devolver las tres juntas en un **vector**, y ahí el llamador las recupera por índice (`resultado[0]` , `resultado[1]` , `resultado[2]`). Esa es la forma que usa la cátedra cuando una función tiene que entregar más de un valor — no un objeto.

6. 🗝️ Las variables de adentro no se ven desde afuera

Todo lo que declararás dentro de una función **vive y muere ahí**. Es el mismo ámbito de bloque de la unidad 03, aplicado a funciones.

```
const power = (voltage, current) => {
  const result = voltage * current
  return result
}

console.log(power(380, 4.2))
console.log(result)
```

```
1596
ReferenceError: result is not defined
```

Y eso es bueno. Significa que podés usar `result` , `i` o `total` adentro de una función sin miedo a pisar una variable del programa principal.

🔑 **La regla:** una función trabaja con **sus parámetros** y sus variables locales. Si necesita un dato, se lo pasás por parámetro — no lo lee de afuera.

7. JSDoc: documentar la función

En la cátedra toda función lleva un comentario **JSDoc**. No es decoración: es lo que hace que alguien —o vos en tres semanas— entienda qué hace sin leer el cuerpo. Se corrige.

```
/**
 * Calcula la desviación porcentual de una medición respecto del valor nominal.
 * @param {number} measured - Valor medido
 * @param {number} nominal - Valor nominal de referencia
 * @returns {number} Desviación en porcentaje, positiva o negativa
 */
const deviation = (measured, nominal) => (measured - nominal) / nominal * 100
```

Las tres partes:

PARTE	QUÉ VA
Primera línea	Qué hace la función, en una oración
<code>@param</code>	Uno por cada parámetro: tipo, nombre y para qué sirve
<code>@returns</code>	Tipo y qué representa el valor devuelto

Los tipos van entre llaves: `{number}`, `{string}`, `{boolean}`. Desde la unidad 07 vas a ver `{number[]}` para vectores.

 **En VSCode se paga solo:** cuando escribís `deviation(` te muestra la descripción y los parámetros en un cartelito. Probalo.

8. Buenas prácticas

PRÁCTICA	POR QUÉ
Un nombre que diga qué hace	<code>deviation</code> sí; <code>calc</code> , <code>f</code> , <code>hacer</code> no
Una sola tarea por función	Si el nombre necesita un "y", son dos funciones
Corta	Si no entra en la pantalla, se puede partir
Sin leer variables de afuera	Todo lo que necesita, por parámetro
Siempre JSDoc	Se corrige
Verbo al principio	<code>calculateAverage</code> , <code>isEven</code> , <code>printHeader</code>

Sobre el último punto, dos convenciones que vas a ver en todos lados:

- Las funciones que **devuelven un booleano** empiezan con `is` o `has`: `isEven`, `isWithinRange`

- Las que **calculan** empiezan con un verbo: `calculate...`, `convert...`, `generate...`

9. 🎲 `Math.random()`

Devuelve un número decimal aleatorio **entre 0 y 1**, incluyendo el 0 pero **nunca el 1**.

```
console.log(Math.random())
console.log(Math.random())
```

```
0.4712...
0.9038...
```

Cada ejecución da otro número. Así solo no sirve de mucho: nadie mide tensiones entre 0 y 1. Hay que **estirarlo** al rango que te interesa.

10. ★ Construir `rndInt(min, max)`

La función más útil del curso. Genera un número entero aleatorio entre `min` y `max`, **incluyendo los dos extremos**. La vamos a usar de acá hasta la última unidad.

Se arma en tres pasos.

Paso 1 – estirar el rango

`Math.random()` da `[0, 1)`. Si querés `max - min + 1` valores posibles, multiplicás:

```
Math.random() * (max - min + 1)
```

Para `min = 1` y `max = 6`, eso da un decimal entre `0` y `6` (sin llegar a 6).

🤔 **¿Por qué el `+ 1`?** Porque los extremos se cuentan los dos. Del 1 al 6 hay **seis** números, no cinco: `6 - 1 = 5`, y le sumás 1.

Paso 2 – recortar los decimales

`Math.floor()` redondea **para abajo**: `Math.floor(3.9)` es `3`.

```
Math.floor(Math.random() * (max - min + 1))
```

Ahora tenés un entero entre `0` y `max - min`. Para el dado: entre `0` y `5`.

Paso 3 – correr el rango

Le sumás `min` para que empiece donde tiene que empezar:

```
Math.floor(Math.random() * (max - min + 1)) + min
```

Entre `1` y `6`. 🎲

La función, completa

```
/**
 * Genera un número entero aleatorio en el rango [min, max].
 * @param {number} min - Valor mínimo del rango (inclusive)
 * @param {number} max - Valor máximo del rango (inclusive)
 * @returns {number} Entero aleatorio entre min y max
 */
const rndInt = (min, max) => Math.floor(Math.random() * (max - min + 1)) + min
```

📌 **Copiala tal cual.** Es la versión de la cátedra: la vas a reusar en todos los TP y en los parciales, siempre con este nombre y esta firma.

📌 **Acordate de la sección 2:** `rndInt` **no es una función matemática.** Con los mismos argumentos devuelve valores distintos, y la definición de función exige un solo valor por cada elemento del dominio. En programación se puede; en matemática, no.

10.1 ⚠ El error clásico: olvidarse el `+ 1`

Sin él, `max` **nunca sale:**

```
// ❌ Genera entre min y max - 1
Math.floor(Math.random() * (max - min)) + min
```

Con `rndInt(1, 6)` nunca sacarías un 6. Y es difícil de notar: el programa anda, los números parecen razonables, y sólo falta uno de todos los posibles.

11. 🏭 Simular mediciones

Con `rndInt()` podés generar los datos en vez de tipearlos, que es lo que hace posible probar un programa con cien casos:

```
const NOMINAL_VOLTAGE = 380

/**
 * Genera una medición de tensión simulada dentro de un rango razonable.
 * @returns {number} Tensión simulada, en volt
 */
const simulateMeasurement = () => rndInt(355, 405)

for (let i = 1; i <= 5; i++) {
  console.log(`Medición ${i}: ${simulateMeasurement()} V`)
}
```

Una función **puede no tener parámetros**: los paréntesis van igual, vacíos.

12. ✂ El pago de la promesa

El informe del TP de la unidad 05, con funciones. Compará el antes y el después.

```
const NOMINAL_VOLTAGE = 380
const TOLERANCE = 0.05

/**
 * Indica si una medición está dentro de la tolerancia admitida.
 * @param {number} measured - Tensión medida, en volt
 * @returns {boolean} true si está dentro de la tolerancia
 */
const isWithinRange = (measured) =>
  measured >= NOMINAL_VOLTAGE * (1 - TOLERANCE) &&
  measured <= NOMINAL_VOLTAGE * (1 + TOLERANCE)

/**
 * Calcula la desviación porcentual respecto de la tensión nominal.
 * @param {number} measured - Tensión medida, en volt
 * @returns {number} Desviación en porcentaje
 */
const deviation = (measured) => (measured - NOMINAL_VOLTAGE) / NOMINAL_VOLTAGE * 100

/**
 * Muestra una línea de informe para una medición.
 * @param {number} index - Número de medición
 * @param {number} measured - Tensión medida, en volt
 */
const printLine = (index, measured) => {
  const mark = isWithinRange(measured) ? ' ' : ' ←'
  console.log(`Medición ${index}: ${measured.toFixed(1)} V  ${deviation(measured).toFixed(2)} %`)
}
```

Ahora el programa principal se lee como una oración:

```
for (let i = 1; i <= 5; i++) {  
  printLine(i, simulateMeasurement())  
}
```

🔑 **Eso es lo que ganás.** El programa principal cuenta *qué* hace; cada función cuenta *cómo*. Y si mañana cambia la tolerancia, hay **un solo lugar** donde tocar.

13. 🐛 Errores comunes

ERROR	SÍNTOMA	SOLUCIÓN
Olvidar el <code>return</code>	La función devuelve <code>undefined</code>	Devolvé el resultado
No usar lo que devuelve	La cuenta se hace y se tira	Guardala o imprimila
Faltan argumentos	El resultado da <code>NaN</code>	Contá los parámetros
Argumentos en el orden equivocado	Resultado incorrecto sin error	El orden de la declaración manda
Usar una variable local afuera	<code>ReferenceError</code>	Devuelvela con <code>return</code>
Leer variables de afuera	La función deja de ser reutilizable	Pasala por parámetro
<code>rndInt</code> sin el <code>+ 1</code>	<code>max</code> no sale nunca	<code>(max - min + 1)</code>
Función sin JSDoc	Se corrige	Escribilo antes del código
Declarar y no llamar	No pasa nada	Declarar $\neq$ ejecutar

14. 📋 Resumen

CONCEPTO	SINTAXIS
Arrow function con cuerpo	<code>const f = (a, b) => { return a + b }</code>
Arrow function corta	<code>const f = (a, b) => a + b</code>
Sin parámetros	<code>const f = () => 42</code>
Lamarla	<code>f(3, 4)</code>
Booleana	<code>const isEven = (n) => n % 2 === 0</code>
Aleatorio en <code>[min, max]</code>	<code>Math.floor(Math.random() * (max - min + 1)) + min</code>

```
/**
 * Qué hace la función, en una oración.
 * @param {number} nombre - Para qué sirve
 * @returns {number} Qué representa lo que devuelve
 */
```

15. 🕒 Para la próxima clase

En la **unidad 07** llegan los **vectores**: por fin un lugar donde **guardar** las mediciones, en vez de procesarlas de a una y perderlas.

Todo lo de hoy se junta ahí: vas a generar vectores con `rndInt()`, recorrerlos con los bucles de la unidad 05, y escribir una función por cada cosa que quieras calcular sobre ellos.

Para llegar preparado:

1. ✅ Hacé el TP — el problema 7 es el que más se parece a lo que viene
2. ✅ Tené `rndInt()` escrita **de memoria**, con su JSDoc
3. ✅ Repasá los tres patrones de la unidad 05: en la 07 se escriben **dentro de funciones**

📎 Material de la unidad

- **Presentación de clase** (<https://github.com/italijancic/pc-2026/blob/main/unidades/06-funciones/presentacion.pdf>)
- **Trabajo Práctico** (<https://github.com/italijancic/pc-2026/blob/main/unidades/06-funciones/tp.pdf>)
- **Ejemplos de la clase** (<https://github.com/italijancic/pc-2026/tree/main/unidades/06-funciones/ejemplos>)